

DLDCALab 3: Performance

Week 2

August 26, 2025

Introduction

In this week's lab, you shall be learning how to optimize code by exploiting microarchitecture. You shall learn to find the "hotspots" in your code and tune them to be faster.

Necessary tools

We shall be using gcc and g++ in order to compile C and C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ gcc
gcc: fatal error: no input files
compilation terminated.
$ g++
g++: fatal error: no input files
compilation terminated.
```

Structure of submission/templates

```
your-roll-no/
|- task1/
   |- cache-size-pred.cpp (TODO)
   |- data.txt (TODO)
   |- time_vs_size_of_buffer.png (TODO)
```

```
| - plotter.py  
| - format.txt  
| - time_access.cpp
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

This assignment consists of one task. It is expected to be submitted in-lab. The general goal is to explore properties of caches.

Task 1: Cache size prediction

The size of a cache is a system dependent attribute that can affect performance of a program (run time). The effect of cache size on performance can be exploited to infer the cache size programatically.

Firstly run `lscpu` and explore the output to see various parameters of your cpu and the cache by extension. Particularly, note the cache sizes of the various levels of the cache (L1, L2, LLC-Last level cache/L3). The goal of this task is to write a program 'cache-size-pred.cpp' to generate information that can be used to predict the size of the cache.

The basic concept behind the task is the idea that when accessing an array repeatedly, the amount of time it takes per access is dependant on what portion of the array fits in the cache hierarchy. To understand why this is true, assume an array that is slightly smaller in size than the L1 cache. In this case, the entire array fits in the L1 cache and the subsequent access to all elements will be L1 hits which makes the average access time very low. In contrast if the array is significantly bigger than L1, a portion of the accesses will be L1 misses which increases the average access time for the array.

Thus write a cpp program to do the same as mentioned above. You must redirect the output of this program into a text file data.txt. You are expected to plot the average time / memory access for various array sizes to figure out the size of the cache. A plotting script (use python matplotlib or gnuplot) can be used for the

same. Make sure a single run of your program prints out all the relevant data you would need to plot.

Points to note:

1. Every access should be to a new cache line if you want to sample the cache properly, if not every spatially local access will be an L1 hit and thus dilute your averages which makes noise more significant.
2. `lscpu` will tell you the total size of L1/L2 as well as the number of instances, the size of L1/L2 per core is then the total size/number of instances. For example, an output of L1d: 768 KiB (12 instances) implies the size of L1D is $768/12 = 64\text{KB}$ per core
3. You should take the average of a large number of readings to get a good number around 100 is suggested
4. You can refer to the file `time_access.cpp` to see how you can time a memory access. Do not time each memory access rather measure time for accessing the entire buffer or chunks of it using the same method.
5. Make sure to compile the file with the `'-O0'` flag to make sure the compiler does not optimise your code, which may have unintended consequences.
6. During measurements please close background processes to prevent noise interference.

```
$ g++ cache-size-pred.cpp -o cache-size-pred -O0
$ ./cache-size-pred > data.txt
$ python3 plotter.py --filename data.txt
```

You have been provided a plotter (python script) which takes a filename argument and plots the data in the file. Note that the data dump must have a specific format for the plotter to work properly and thus use the `'format.txt'` file to write your cpp file to dump the data in the appropriate format. Optionally you can also update the python script to read data from a file of a different format.